



南開大學
Nankai University

计算机学院
并行程序设计实验报告

Lab01 体系结构相关编程实验

姓名：请在此处填写

学号：请在此处填写

专业：计算机科学与技术

2026 年 4 月 3 日

目录

1 实验概述	2
1.1 实验目的	2
1.2 研究问题与待验证假设	2
1.3 实验完成情况	2
2 实验环境	2
2.1 软硬件平台	2
2.2 测试方法	3
2.3 实验流程与数据处理	3
2.4 可复现性说明	3
3 实验设计与实现	3
3.1 矩阵与向量内积	3
3.1.1 算法设计	3
3.1.2 关键实现	4
3.1.3 规模设置说明	4
3.1.4 工作集估算	4
3.2 n 个数求和	4
3.2.1 算法设计	4
3.2.2 规模设置说明	5
3.2.3 工作集估算	5
3.3 编译与运行	5
4 实验结果与分析	6
4.1 矩阵与向量内积结果	6
4.1.1 结果分析	7
4.2 n 个数求和结果	7
4.2.1 结果分析	9
4.3 综合结论	9
5 VTune 剖析	9
5.1 矩阵题: Hotspots 与 Memory Access	9
5.1.1 Hotspots 结果	10
5.1.2 Memory Access 结果	10
5.1.3 矩阵题 VTune 结论	10
5.2 求和题: Hotspots 与 Microarchitecture Exploration	11
5.2.1 Hotspots 结果	11
5.2.2 Microarchitecture Exploration 结果	11
5.2.3 求和题 VTune 结论	12
6 Godbolt 汇编分析	12
6.1 分析方法	12
6.2 矩阵题汇编分析	12
6.2.1 col_naive 与 row_cache	12
6.2.2 row_cache 与 row_cache_u4	13
6.3 求和题汇编分析	13
6.3.1 sum_naive 与 sum_dual / sum_unroll14	13
6.3.2 sum_unroll14 与 sum_pairwise	13
6.4 Godbolt 分析结论	13

1 实验概述

1.1 实验目的

本实验对应 Lab01「体系结构相关编程」，报告组织与实验设计遵循课程指导书与配套课件中关于实验环境、实验设计、结果呈现和 profiling 分析的要求。实验的目标是围绕 CPU cache 局部性、指令级并行和编译器优化三个主题，完成两类微基准程序的实现、验证与性能测试：

1. 矩阵与向量内积：比较逐列访问的平凡算法与按行访问的 cache 优化算法，并进一步尝试循环展开。
2. n 个数求和：比较单链式累加与多路链式累加，观察减少数据相关后对超标量执行效率的影响。
3. 对比不同编译优化等级（-O0、-O2、-O3）下的运行结果，并结合 VTune 与 Godbolt 对性能来源进行剖析。

1.2 研究问题与待验证假设

结合 Lab1 文档中的题目分析，本报告重点回答以下三个问题：

1. 对矩阵题而言，改变访存顺序是否能显著改善 cache 行为，并且这种优势是否会随着工作集增大而继续扩大？
2. 对求和题而言，多累加器与循环展开是否能有效降低累加依赖链长度，从而在 -O2/-O3 下提高超标量利用率？
3. 编译器优化、算法设计和 profiling 结果之间是否能形成一致的证据链，解释实验中的主要性能差异？

对应地，本文待验证的假设为：矩阵题的主要瓶颈来源于访存模式不匹配，求和题的主要瓶颈来源于串行依赖链，而编译器高优化等级会进一步放大这两类手工优化的收益。

1.3 实验完成情况

结合 Lab01 要求，当前已经完成的内容如下：

- 基础要求：矩阵题完成 `col_naive`、`row_cache`；求和题完成 `sum_naive`、`sum_dual`。
- 进阶要求：完成循环展开版本 `row_cache_u4` 与 `sum_unroll4`。
- 补充算法：实现树形求和 `sum_pairwise`，用于对比更多算法设计思路。
- 编译器优化对比：已经在 -O0、-O2、-O3 三种编译优化等级下完成基准测试。
- profiling 分析：已经完成矩阵题的 VTune Hotspots / Memory Access 分析、求和题的 VTune Hotspots / Microarchitecture 分析，以及两组算法的 Godbolt 汇编对比分析。

2 实验环境

2.1 软硬件平台

根据课程要求，实验部分需要明确说明软硬件平台参数，尤其是 CPU 型号、核心数、主频、各级 cache 容量和编译选项。本实验所使用的平台信息如表1所示，其中 CPU 型号、核心数、内存和 cache 容量依据本机任务管理器与实验环境截图整理得到。

项目	配置
操作系统	Microsoft Windows 10.0.26200.8037
编译器	g++ 15.2.0 (MinGW-W64 x86_64-ucrt-posix-seh)
LaTeX 编译器	XeLaTeX 2025
构建方式	make o0 / make o2 / make o3 / make vtune
CPU 型号	Intel Core Ultra 7 155H
CPU 基准频率	3.80 GHz
物理核心数 / 逻辑线程数	16 / 22
L1/L2/L3 Cache	1.6 MB / 18.0 MB / 24.0 MB
内存容量	31.5 GB
存储设备	NVMe SSD
profiling 工具	Intel VTune Profiler、Godbolt (Compiler Explorer)

表 1: 实验平台信息

2.2 测试方法

本实验采用项目中的微基准程序进行性能测试。测试程序会先完成正确性验证，再对目标算法重复执行若干次并统计总耗时。计时函数使用 `std::chrono::high_resolution_clock` 封装得到的秒级时间差，测试策略遵循“对短程序重复执行、延长计时间隔以提高测量精度”的基本原则。输出字段包括算法名、问题规模、重复次数、总耗时以及相对基础算法的加速比。

对任意算法，其加速比定义为：

$$\text{speedup} = \frac{T_{\text{baseline}}}{T_{\text{current}}} \quad (1)$$

其中，矩阵题的基准算法为 `col_naive`，求和题的基准算法为 `sum_naive`。

2.3 实验流程与数据处理

参考以往实验资料中的做法，本次报告将实验过程拆分为“程序实现、批量执行、结果整理、图表呈现”四个阶段：

1. 源程序负责实现算法、完成正确性校验，并输出结构化测试结果；
2. 通过 `Makefile` 分别生成 `-00`、`-02`、`-03` 三组可执行文件；
3. 将程序输出整理到 `RESULTS.md`，再进一步抽取为表格与折线图；
4. 在报告中同时保留“原始数值表”和“趋势图”，避免只展示终端截图或孤立数据点。

这种组织方式的优点在于：一方面能让每个结论都有对应原始数据支撑，另一方面也便于后续继续接入 `VTune`、`Godbolt` 或 `CSV` 脚本做更细粒度分析。

2.4 可复现性说明

为了保证结果具有可重复性，当前程序已经具备以下基础条件：

- 每种算法在正式计时前都会先进行一次正确性验证；
- 对不同规模使用固定的重复次数，以避免单次运行时间过短导致计时误差过大；
- 同一组实验中的不同算法使用相同输入数据，保证对比公平。

若后续需要进一步提高实验严谨性，可再补充固定随机种子、CPU 绑核、关闭动态睿频、导出 `CSV` 原始数据等措施。

3 实验设计与实现

3.1 矩阵与向量内积

设矩阵 $B \in \mathbb{R}^{n \times n}$ 、向量 $a \in \mathbb{R}^n$ ，计算目标为：

$$\text{sum}[i] = \sum_{j=0}^{n-1} B[j][i] \cdot a[j], \quad 0 \leq i < n \quad (2)$$

由于 C/C++ 中二维数组按行主序存储，若按列访问矩阵，会造成跨 `cache line` 的非连续读；若按行访问，则更符合空间局部性。

3.1.1 算法设计

1. `col_naive`：固定列索引 i ，内层枚举行索引 j ，直接按数学定义实现。
2. `row_cache`：固定行索引 j ，把当前 $a[j]$ 缓存在寄存器中，并顺序扫描该行全部元素，降低跨行跳跃访问带来的 `cache miss`。
3. `row_cache_u4`：在 `row_cache` 基础上对内层循环做四路展开，减少循环控制与索引更新开销。

Algorithm 1 矩阵列内积的 `cache` 优化思路

Input: 行主序存储矩阵 B ，向量 a ，输出数组 `sum`，规模 n

Output: 输出每一列与向量 a 的内积

```

1: 将 sum[0...n-1] 置零
2: for  $j = 0$  to  $n-1$  do
3:    $aj \leftarrow a[j]$ 
4:    $base \leftarrow j \times n$ 
5:   for  $i = 0$  to  $n-1$  do
6:      $\text{sum}[i] \leftarrow \text{sum}[i] + B[base + i] \times aj$ 
7:   end for
8: end for
9: return sum
```

3.1.2 关键实现

矩阵题中按行访问的核心实现

```

1 void row_cache(const Matrix& b, const Vector& a, Vector& sum, std::size_t n) {
2     std::fill(sum.begin(), sum.end(), 0.0);
3     for (std::size_t j = 0; j < n; ++j) {
4         const double aj = a[j];
5         const std::size_t base = j * n;
6         for (std::size_t i = 0; i < n; ++i) {
7             sum[i] += b[base + i] * aj;
8         }
9     }
10 }

```

3.1.3 规模设置说明

矩阵题选择 $n = 128, 256, 512, 1024$ 四个规模，考虑如下：

1. 这四组规模覆盖了从较小工作集到较大工作集的增长过程，足以观察 cache 局部性对性能的影响是否随规模扩大而增强；
2. 随着 n 增大，矩阵数据量按平方增长，更容易放大“按列访问”和“按行访问”的访存差异；
3. 重复次数采用分段递减策略，目的是在保证总耗时可接受的同时，让不同规模下的测量仍然具有稳定性。

3.1.4 工作集估算

为了将问题规模与 cache 行为联系起来，表2 给出了矩阵题主要数据结构的大致工作集大小。这里按双精度浮点数 8 Byte 估算，矩阵 B 占用 $8n^2$ Byte，向量 a 与输出数组 sum 共占用 $16n$ Byte。可以看出，测试规模覆盖了从百 KiB 到数 MiB 的区间，这有助于观察工作集逐步逼近私有 cache、共享 cache 乃至主存时的性能变化。

规模 n	矩阵 B	向量与结果数组	总工作集
128	128 KiB	2 KiB	130 KiB
256	512 KiB	4 KiB	516 KiB
512	2.00 MiB	8 KiB	2.01 MiB
1024	8.00 MiB	16 KiB	8.02 MiB

表 2: 矩阵题工作集规模估算

3.2 n 个数求和

求和实验关注的不是算法复杂度，而是累加过程中的数据相关。单链式累加存在严格的串行依赖，而多路累加可把长依赖链拆成多条较短的依赖链，更有利于处理器发掘指令级并行。

3.2.1 算法设计

1. **sum_naive**: 使用单个累加器顺序累加，作为基准实现。
2. **sum_dual**: 使用两个累加器分别处理偶数位与奇数位，减少单一路径的相关长度。
3. **sum_unroll4**: 采用四个累加器并配合四路循环展开。
4. **sum_pairwise**: 使用树形规约思路，两两相加后写回数组，体现不同设计思路，但会引入额外访存。

Algorithm 2 四路展开求和

Input: 输入数组 $a[0 \dots n-1]$

Output: 数组所有元素之和

```

1:  $s_0, s_1, s_2, s_3 \leftarrow 0$ 
2:  $i \leftarrow 0$ 
3: while  $i + 3 < n$  do
4:    $s_0 \leftarrow s_0 + a[i]$ 
5:    $s_1 \leftarrow s_1 + a[i + 1]$ 
6:    $s_2 \leftarrow s_2 + a[i + 2]$ 
7:    $s_3 \leftarrow s_3 + a[i + 3]$ 
8:    $i \leftarrow i + 4$ 

```

```

9: end while
10: while  $i < n$  do
11:    $s_0 \leftarrow s_0 + a[i]$ 
12:    $i \leftarrow i + 1$ 
13: end while
14: return  $(s_0 + s_1) + (s_2 + s_3)$ 

```

求和实验中的四路展开实现

```

1 double sum_unroll4(const Vector& a) {
2     double sum0 = 0.0, sum1 = 0.0, sum2 = 0.0, sum3 = 0.0;
3     std::size_t i = 0;
4     for (; i + 3 < a.size(); i += 4) {
5         sum0 += a[i];
6         sum1 += a[i + 1];
7         sum2 += a[i + 2];
8         sum3 += a[i + 3];
9     }
10    for (; i < a.size(); ++i) {
11        sum0 += a[i];
12    }
13    return (sum0 + sum1) + (sum2 + sum3);
14 }

```

3.2.2 规模设置说明

求和题选择 2^{10} 、 2^{14} 、 2^{18} 、 2^{22} 四组输入规模，主要是为了让数据规模跨越多个数量级：

1. 小规模时，处理器更可能受益于减少依赖链长度带来的指令级并行提升；
2. 大规模时，数据访问的带宽与 cache 行为会逐渐成为主导因素，可以观察展开收益是否衰减；
3. 以 2^k 形式设置规模，也便于把实验现象与 cache 容量、对齐和规约层数联系起来分析。

3.2.3 工作集估算

对于求和题，输入数组大小为 $8n$ Byte。表3显示，测试规模从 8 KiB 跨越到 32 MiB，既包含很可能完全驻留于较低层 cache 的小数组，也包含必须频繁访问更高层 cache 甚至主存的大数组。这样的设置有利于观察“依赖链优化收益”与“内存访问开销”之间的主导关系如何随规模改变。

规模 n	输入数组大小
2^{10}	8 KiB
2^{14}	128 KiB
2^{18}	2 MiB
2^{22}	32 MiB

表 3: 求和题工作集规模估算

3.3 编译与运行

项目提供统一的 **Makefile**，分别使用如下命令生成不同优化等级的可执行文件：

构建命令

```

1 make o0
2 make o2
3 make o3

```

对应的测试程序如下：

- bin/matrix_dot_o0.exe、bin/matrix_dot_o2.exe、bin/matrix_dot_o3.exe
- bin/sum_bench_o0.exe、bin/sum_bench_o2.exe、bin/sum_bench_o3.exe

4 实验结果与分析

4.1 矩阵与向量内积结果

矩阵题测试规模为 $n = 128, 256, 512, 1024$ ，结果如表4 所示。

优化等级	规模 n	col_naive/s	row_cache/s	row_cache_u4/s	最优加速比
-00	128	0.035911	0.032405	0.032201	1.115
-00	256	0.045614	0.038836	0.038891	1.175
-00	512	0.066748	0.038642	0.038735	1.727
-00	1024	0.232900	0.036803	0.044080	6.328
-02	128	0.005892	0.002102	0.001640	3.592
-02	256	0.005733	0.002111	0.001892	3.030
-02	512	0.006746	0.002036	0.001957	3.447
-02	1024	0.048577	0.004488	0.003531	13.758
-03	128	0.005127	0.001258	0.000972	5.274
-03	256	0.005754	0.001380	0.001369	4.203
-03	512	0.007091	0.001192	0.001243	5.951
-03	1024	0.043438	0.003575	0.003100	14.011

表 4: 矩阵列内积实验结果

图4.1 展示了 -03 下不同规模的相对加速比变化趋势。

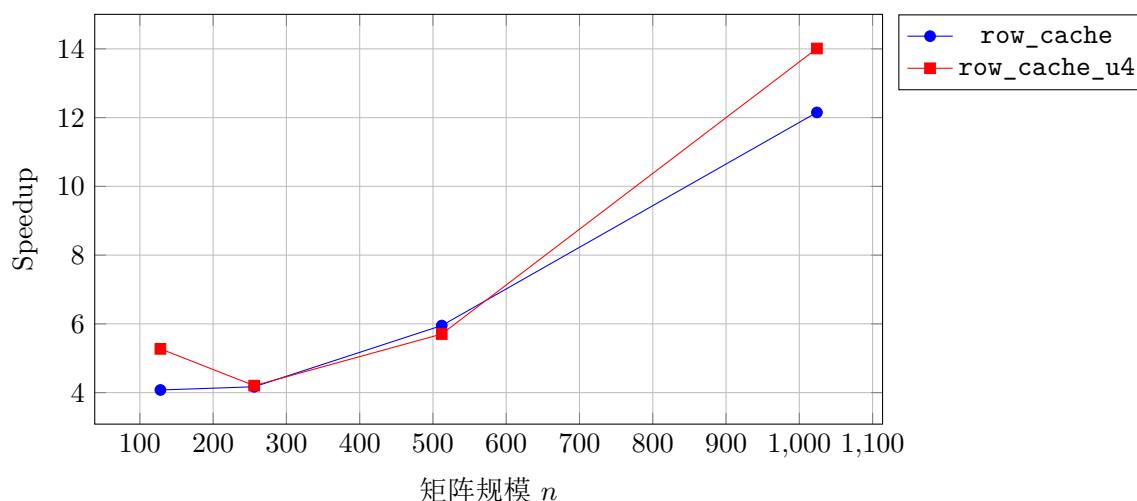


图 4.1: -03 下矩阵题相对 col_naive 的加速比

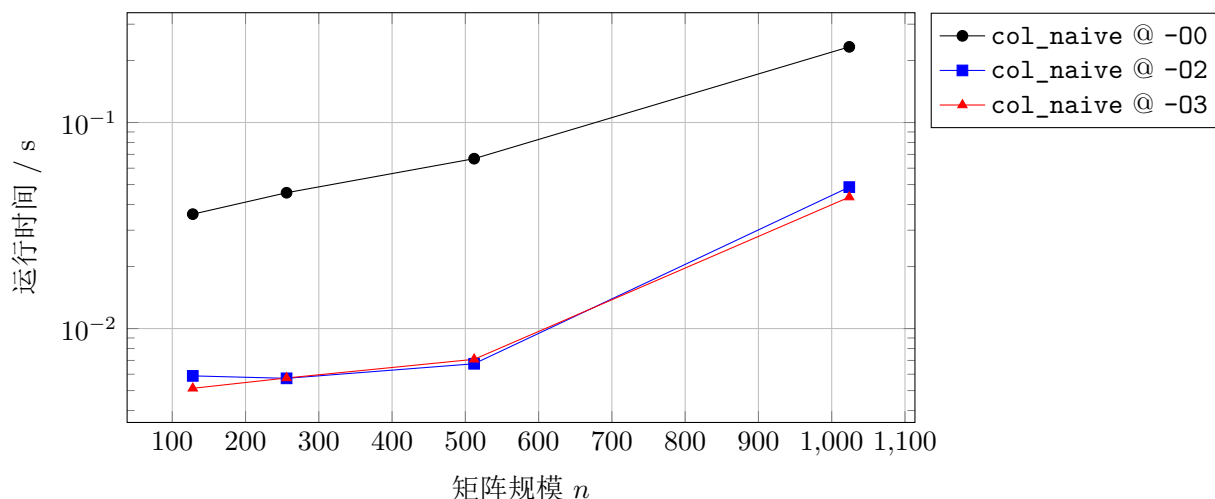


图 4.2: 矩阵题中 col_naive 在不同优化等级下的运行时间

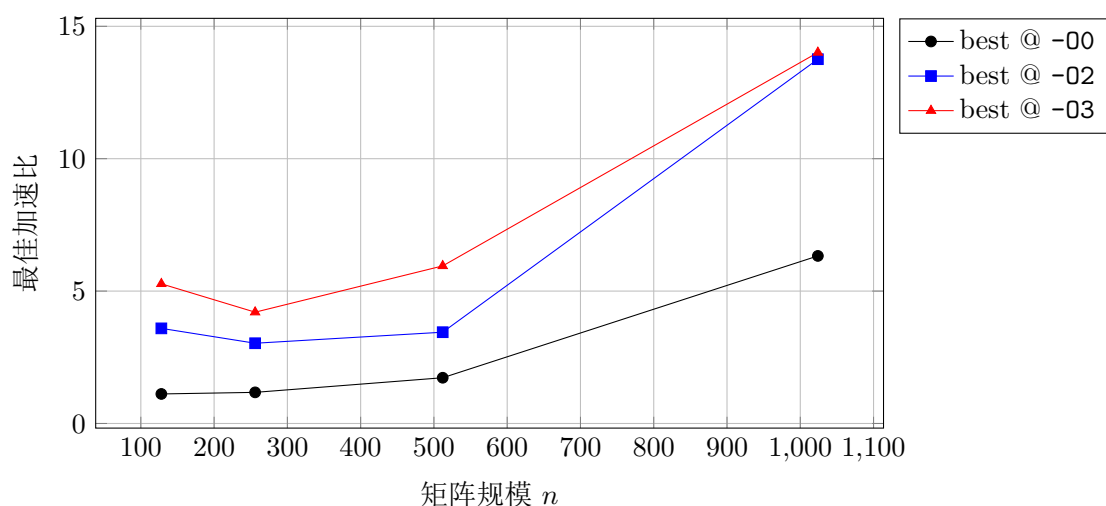


图 4.3: 不同优化等级下矩阵题的最佳加速比

4.1.1 结果分析

1. row_cache 在所有规模和优化等级下均明显优于 col_naive。这一现象与表2 所示工作集增长趋势相一致：随着矩阵总工作集从 130 KiB 扩大到 8.02 MiB，按列访问造成的跨步读代价被不断放大，而按行顺序访问仍能持续受益于 cache line 的空间局部性。
2. 图4.1 和图4.3 共同说明，矩阵题的收益来源具有明显的“双重放大”特征：一方面，规模越大，cache 友好访问带来的优势越强；另一方面，编译器优化越高，算法层面的优势越容易被转化为实际加速。特别是 $n = 1024$ 时，-03 下 row_cache_u4 的加速比达到 14.011 倍。
3. row_cache_u4 在多数情况下优于 row_cache，但并非严格单调领先。例如 -03、 $n = 512$ 时，row_cache 略优于 row_cache_u4。这表明循环展开的收益属于“次一级优化”：它主要减少循环控制开销，但最终效果仍受寄存器分配、指令调度和自动向量化策略影响。
4. 图4.2 表明，编译器优化对基线算法本身也有显著提升作用。以 col_naive 为例，-03 在大规模下的运行时间明显低于 -00；然而即便如此，不良访存模式并未被根本修复，因此算法设计仍然是决定性因素。
5. 综合来看，矩阵题最核心的科学结论是：在行主序存储条件下，访存顺序比单纯的局部代码展开更重要；循环展开的价值在于进一步压缩已经“访存友好”的循环体，而不是替代访存优化。

4.2 n 个数求和结果

求和实验的规模分别为 2^{10} 、 2^{14} 、 2^{18} 、 2^{22} 。测试结果如表5 所示。

优化等级	规模 n	sum_naive/s	sum_dual/s	sum_unroll4/s	sum_pairwise/s	最优加速比
-00	1024	0.041079	0.070381	0.064958	0.190056	1.000
-00	16384	0.128563	0.223203	0.210417	0.665581	1.000
-00	262144	0.338992	0.566687	0.474945	1.428311	1.000
-00	4194304	0.729098	1.257167	1.137982	3.279034	1.000
-02	1024	0.008649	0.004537	0.002311	0.006870	3.742
-02	16384	0.039790	0.016440	0.007876	0.029359	5.052
-02	262144	0.076637	0.045045	0.020669	0.419883	3.708
-02	4194304	0.214646	0.164202	0.163617	1.111949	1.312
-03	1024	0.009025	0.004388	0.002246	0.007210	4.018
-03	16384	0.031813	0.015477	0.007777	0.029042	4.091
-03	262144	0.095107	0.045808	0.021920	0.377787	4.339
-03	4194304	0.190002	0.137766	0.140909	0.993874	1.379

表 5: n 个数求和实验结果

图4.4 展示了 -03 下三种改进算法相对于 sum_naive 的加速比。

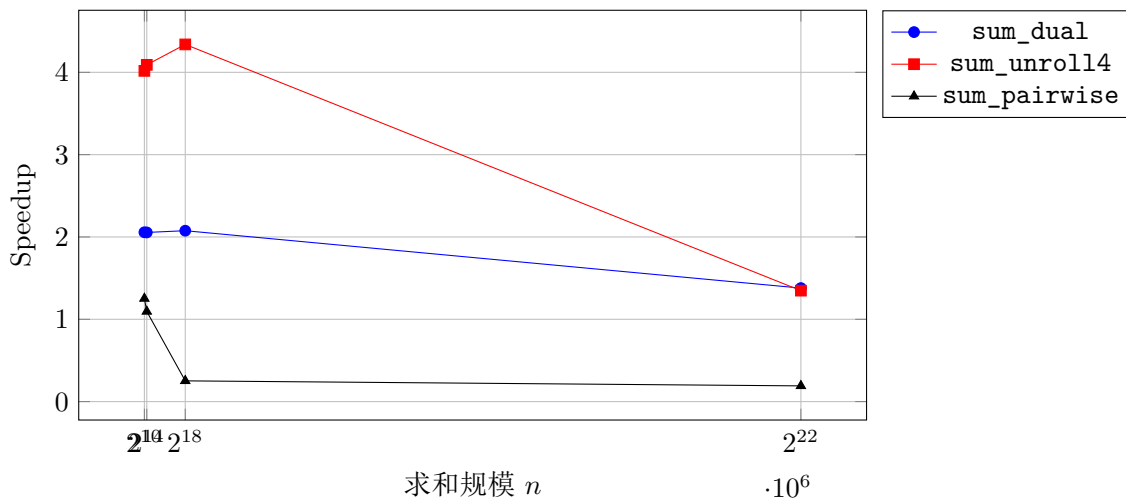


图 4.4: -03 下求和题相对 sum_naive 的加速比

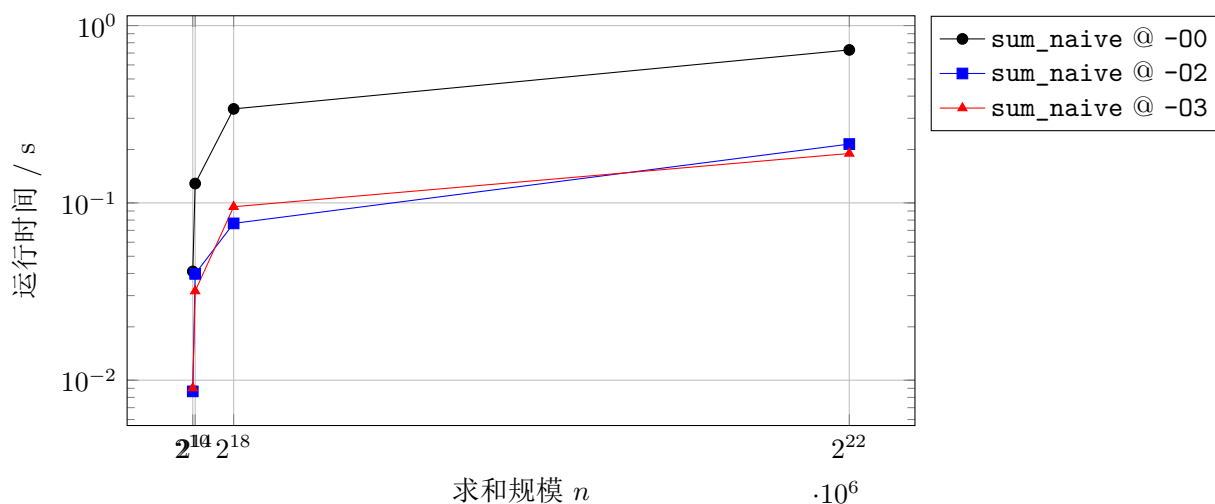


图 4.5: 求和题中 sum_naive 在不同优化等级下的运行时间

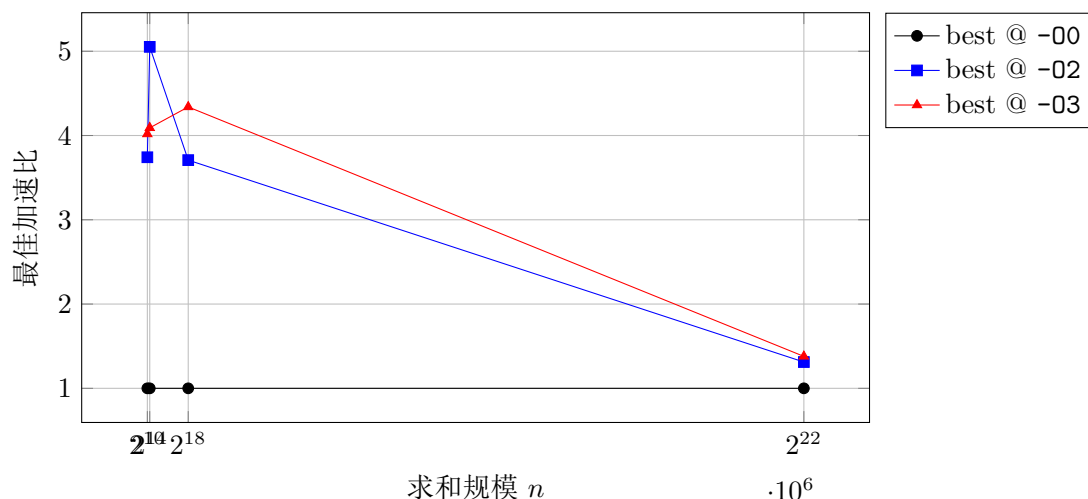


图 4.6: 不同优化等级下求和题的最佳加速比

4.2.1 结果分析

1. 在 -00 下, `sum_dual` 和 `sum_unroll14` 不仅没有获得加速, 反而普遍慢于 `sum_naive`。这说明若编译器几乎不进行寄存器分配和向量化优化, 多累加器结构自身也会引入更多临时变量和控制开销。
2. 当优化等级提升到 -02 和 -03 后, 多路累加的优势迅速显现。图4.4 和图4.6 显示, 在 2^{10} 至 2^{18} 的规模区间内, `sum_unroll14` 可稳定获得约 4 倍加速, 说明“缩短依赖链 + 减少循环控制”在这一工作集范围内能够被处理器和编译器充分利用。
3. `sum_unroll14` 在中等规模上表现最佳。例如 -03、 $n = 262144$ 时相对 `sum_naive` 的加速比达到 4.339。结合表3 可知, 此时输入数组大小约为 2 MiB, 已经足以使访存成本不可忽略, 但仍未完全压制指令级并行优化的收益。
4. 当规模达到 2^{22} (32 MiB) 时, `sum_dual` 与 `sum_unroll14` 的优势显著缩小至约 1.35 倍。这说明随着工作集进一步增大, 性能瓶颈逐渐从“单条加法依赖链”向“内存访问与数据搬运”迁移, 处理器已无法仅靠提高指令级并行来维持中等规模下的高加速。
5. `sum_pairwise` 的表现则揭示了“算法结构好看不等于实现高效”。在小规模下它偶尔接近基线, 但在大规模下显著落后, 主要原因是树形规约需要反复写回中间结果, 带来明显的复制与额外访存开销。
6. 图4.5 还说明了编译器优化本身对基线算法就有显著加速作用, 因此评价“手工优化是否有效”时, 必须把编译器选项与算法结构作为一个整体来看, 而不能只比较源码表面形式。

4.3 综合结论

结合两组实验, 可以得到以下结论:

1. 对矩阵题而言, 访存顺序决定了 cache 行为。与数据布局一致的顺序访问能够显著降低访存延迟, 其收益会随问题规模扩大而增强。
2. 对求和题而言, 算法复杂度虽然相同, 但数据相关不同。通过多路累加缩短依赖链, 可以显著提高超标量执行效率; 这一收益在中等工作集上最明显, 在超大工作集上会被访存成本部分抵消。
3. 编译器优化等级会强烈影响手工优化的最终表现。高优化等级不仅能降低基线算法的执行时间, 也能把 cache 友好写法和多累加器结构进一步转化为向量化与更有效的指令调度。
4. VTune 与 Godbolt 的剖析结果与性能表、趋势图能够形成一致证据链, 因此本文的主要结论不仅来自经验观察, 也获得了 profiling 和汇编层面的支持。

5 VTune 剖析

5.1 矩阵题: Hotspots 与 Memory Access

为了进一步验证矩阵题中“按列访问慢、按行访问快”的原因, 本文对 `matrix_dot_vtune.exe` 在参数 $n = 1024$ 下进行了 Hotspots 和 Memory Access 分析。分析目标是回答两个问题:

1. 时间主要消耗在哪个函数、哪一行代码上;
2. 性能差异是否确实与内存访问瓶颈有关。

5.1.1 Hotspots 结果

Hotspots 结果表明, 矩阵题中最主要的热点函数是 `col_naive`, 其 CPU Time 约为 70.793 ms, 占总热点时间的 82.0%; 而 `row_cache_u4` 的 CPU Time 约为 15.489 ms, 仅占 18.0%。进一步查看源码热点行可知, `col_naive` 的主要开销集中在按列访问矩阵的语句

```
acc += b[idx(j, i, n)] * a[j];
```

(3)

而 `row_cache_u4` 的热点位于按行顺序访问的展开循环语句

```
sum[i] += b[base + i] * a[j];
```

(4)

这与前文性能测试中 `col_naive` 明显慢于 `row_cache` 和 `row_cache_u4` 的结果是一致的。

函数	CPU Time	占比
<code>col_naive</code>	70.793 ms	82.0%
<code>row_cache_u4</code>	15.489 ms	18.0%

表 6: VTune Hotspots 下矩阵题核心函数对比

5.1.2 Memory Access 结果

Memory Access 分析进一步给出了与访存行为有关的硬件证据。整体结果显示, 本次运行的 **Memory Bound** 为 25.0% pipeline slots, 而 **DRAM Bandwidth Bound** 为 0.0%。这说明矩阵题的瓶颈并不是主存带宽被完全打满, 而更可能是由于不良的访存模式带来的 cache / memory latency 开销。

函数级结果表明, `col_naive` 的相关 CPU 时间约为 55.996 ms, 显著高于 `row_cache` 的 4.000 ms 和 `row_cache_u4` 的 3.000 ms。也就是说, 按列访问带来的跨步读取使流水线出现了更明显的内存停顿; 而按行访问后, 处理器能够更有效地利用 cache line 和顺序读带来的局部性, 因此访存相关开销显著下降。

函数	CPU Time	Memory Bound
<code>col_naive</code>	55.996 ms	25.0%
<code>row_cache</code>	4.000 ms	0.0%
<code>row_cache_u4</code>	3.000 ms	0.0%

表 7: VTune Memory Access 下矩阵题核心函数对比

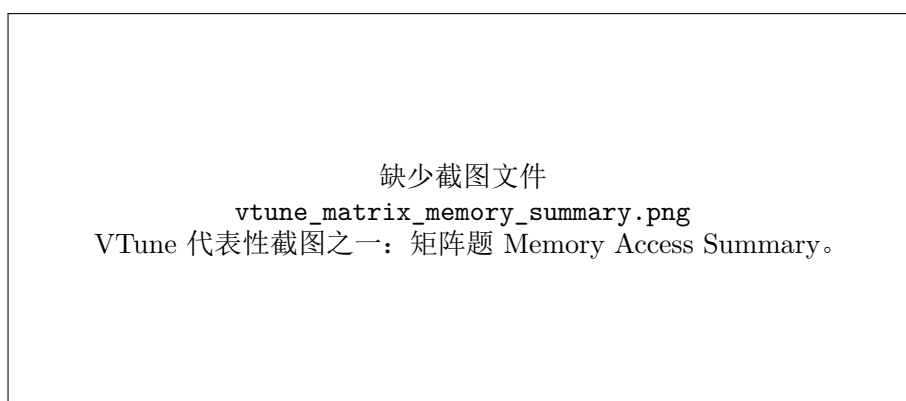


图 5.7: 矩阵题的 VTune Memory Access Summary

5.1.3 矩阵题 VTune 结论

结合 Hotspots 与 Memory Access 两类分析, 可以得到如下结论:

1. `col_naive` 的主要时间开销集中在按列访问矩阵的内层乘法语句上, 说明该语句是程序最主要的性能瓶颈;
2. 矩阵题存在一定的 memory-bound 特征, 但 **DRAM Bandwidth Bound** 为 0.0%, 说明瓶颈主要不是“内存带宽不够”, 而是“访问模式不友好”导致的延迟开销;
3. `row_cache` 和 `row_cache_u4` 通过顺序访问矩阵行显著降低了访存相关停顿, 这与前文性能测试中的大幅加速现象相互印证;
4. `row_cache_u4` 在访存模式已经优化的基础上又进一步减少了循环控制开销, 因此在函数级结果中略优于 `row_cache`。

5.2 求和题：Hotspots 与 Microarchitecture Exploration

为了验证求和题中“多路累加优于单链累加”的原因，本文对 `sum_bench_vtune.exe` 在参数 $n = 262144$ 下进行了 Hotspots 与 Microarchitecture Exploration 分析。分析重点是：

1. 比较 `sum_naive`、`sum_dual`、`sum_unroll4`、`sum_pairwise` 的函数级 CPU 时间；
2. 观察热点源码是否正落在累加语句上；
3. 结合 CPI 与 Top-down 指标判断瓶颈更偏向依赖链、核心执行，还是访存写回。

5.2.1 Hotspots 结果

Hotspots 结果表明，在本次运行中总耗时为 0.583 s，总 CPU 时间为 0.511 s。函数级结果如表 8 所示：`sum_unroll4` 的 CPU 时间仅为 15.809 ms，明显低于 `sum_dual` 的 46.375 ms 和 `sum_naive` 的 62.772 ms，说明四路累加的优化效果最明显。另一方面，`sum_pairwise` 达到 136.124 ms，同时 `std::vector` 构造函数还额外占用了 194.787 ms，这表明当前树形求和实现中的数据复制与中间写回成本很高。

源码级截图进一步给出了直接证据：`sum_naive` 的热点落在 `sum += v;` 上，而 `sum_unroll4` 的热点落在四路展开中的一条局部累加语句上。这与前文 Godbolt 汇编分析的结论一致，即前者主要受单累加器依赖链约束，而后者通过多累加器暴露出更多独立计算链。

函数	CPU Time	CPU 占比
<code>sum_pairwise</code>	136.124 ms	26.7%
<code>sum_naive</code>	62.772 ms	12.3%
<code>sum_dual</code>	46.375 ms	9.1%
<code>sum_unroll4</code>	15.809 ms	3.1%
<code>std::vector::vector</code>	194.787 ms	38.1%

表 8: VTune Hotspots 下求和题核心函数对比

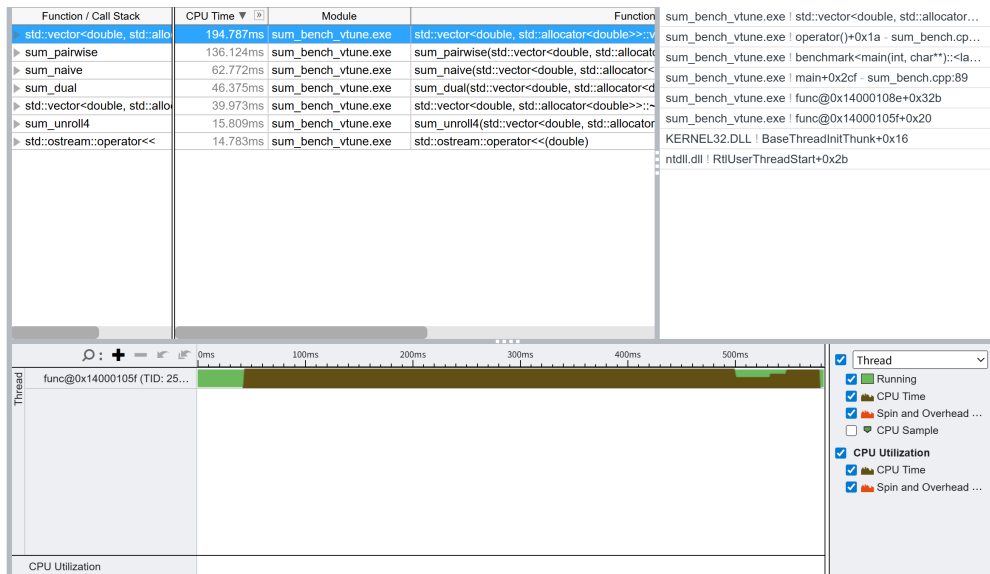


图 5.8: 求和题的 VTune Hotspots Bottom-up 视图

5.2.2 Microarchitecture Exploration 结果

VTune 的微架构分析结果显示：本次运行的 Elapsed Time 为 0.610 s，Instructions Retired 为 3.061×10^9 ，整体 CPI Rate 为 0.767，平均 CPU 频率约 4.4 GHz。进一步从 Top-down 指标看，P-core 上 Retiring 为 20.7%，Front-End Bound 为 5.4%，Back-End Bound 为 82.1%。这说明当前单线程求和程序的主要瓶颈并不在前端取指或分支预测，而更多集中在后端执行阶段，符合“累加依赖链限制吞吐”的预期。

函数级对比更有代表性。对用户函数而言，`sum_naive` 的 CPI 为 0.807，Retiring 为 40.0%；`sum_dual` 的 CPI 降为 0.321，Retiring 提升至 80.0%；`sum_unroll4` 的 CPI 为 0.329。也就是说，多路累加显著降低了每条已退休指令对应的平均周期数，使处理器能够更高效地完成实际计算。相比之下，`sum_pairwise` 虽然 CPI 不高 (0.689)，但其自身 CPU 时间仍然达到 128.991 ms，而且函数级热点中还出现了明显的 `memcpy`，说明它的主要问题不在标量加法吞吐，而在数据复制和重复写回带来的额外代价。

因此，求和题的 VTune 结果与性能表和 Godbolt 汇编分析形成了较强对应关系：`sum_naive` 慢，主要因为单条累加链使后端吞吐受限；`sum_dual` 与 `sum_unroll14` 快，主要因为多累加器降低了依赖链长度；`sum_pairwise` 慢，则主要因为实现方式引入了额外的内存复制与规约写回。

函数	CPU Time	CPI	Retiring	主要现象
<code>sum_naive</code>	70.995 ms	0.807	40.0%	单累加链，后端吞吐受限
<code>sum_dual</code>	41.997 ms	0.321	80.0%	两路累加降低相关性
<code>sum_unroll14</code>	15.999 ms	0.329	12.5%	四路展开，CPU 时间最低
<code>sum_pairwise</code>	128.991 ms	0.689	25.5%	规约写回与复制开销明显
<code>memcpy</code>	131.991 ms	5.387	5.3%	数据复制成为额外热点

表 9: VTune Microarchitecture Exploration 下求和题核心函数对比

5.2.3 求和题 VTune 结论

1. `sum_naive` 的热点确实集中在单条累加语句上，其 CPU 时间显著高于 `sum_dual` 和 `sum_unroll14`；
2. `sum_dual` 与 `sum_unroll14` 通过多累加器降低了串行依赖，函数级 CPI 明显低于 `sum_naive`，说明其指令级并行性更好；
3. 整体 Top-down 结果中 P-core 的 Back-End Bound 达到 82.1%，说明求和题的核心瓶颈主要出现在执行后端，而不是前端取指；
4. `sum_pairwise` 的慢点并不主要是“加法本身效率差”，而是实现过程中引入了大量数据复制和中间结果写回，`memcpy` 成为了显著热点。

6 Godbolt 汇编分析

6.1 分析方法

为了进一步解释性能测试中的差异，本文使用 Godbolt (Compiler Explorer) 观察关键函数在不同优化选项下生成的汇编代码。分析时主要对比以下几组函数：

- 矩阵题：`col_naive`、`row_cache`、`row_cache_u4`
- 求和题：`sum_naive`、`sum_dual`、`sum_unroll14`、`sum_pairwise`

编译器选用 x86-64 GCC，主要考察 -O0、-O3 和 -O3 -march=native 三组编译选项。分析重点包括：

1. 内层循环的地址计算是否简化；
2. 是否减少了循环控制与分支开销；
3. 是否通过多个寄存器暴露更多独立计算链；
4. 编译器是否自动生成了 SSE/AVX 向量指令。

6.2 矩阵题汇编分析

6.2.1 col_naive 与 row_cache

从 -O0 版本可以直接看出，两者都保留了较完整的循环框架和标量浮点操作，但 `col_naive` 的内层循环中需要反复调用索引计算逻辑，表现为“先求 `idx(j,i)`，再读取矩阵元素，再完成乘加”，循环体较长，地址计算也更复杂。相比之下，`row_cache` 在 -O0 下已经显式把当前行首地址 `base` 和当前标量 `a[j]` 缓存在寄存器或栈临时变量中，其核心循环更接近“顺序读取 + 累加写回”的结构。

在 -O3 下，两者都出现了明显优化，但优化方向不同。对于 `col_naive`，编译器已经能够使用 `mulpd/addpd` 对两元素做并行乘加；然而其访存模式本质上仍然是跨行取列，因此虽然计算指令变得更紧凑，内存访问的跨步特点并未改变。对于 `row_cache`，汇编中出现了更接近流式处理的顺序加载与写回，编译器还进一步把相邻元素成组处理，使得核心循环更容易结合 cache line 的顺序访问优势。

在 -O3 -march=native 下，这种差异更加明显。`col_naive` 已经出现 `vbroadcastsd`、`vfmadd231pd` 和 `ymm` 寄存器，说明编译器利用了 AVX/FMA 指令；但由于访存仍然是按列抽取，性能受限因素并没有根本改变。另一方面，`row_cache` 同样生成了 AVX/FMA 指令，而且其内层循环对应连续地址区间上的向量乘加，说明算法层面的 cache 友好布局与编译器层面的向量化优化形成了叠加。因此，Godbolt 上观察到的汇编特征与表 4 中 `row_cache` 和 `row_cache_u4` 明显优于 `col_naive` 的实验结果是一致的。

6.2.2 row_cache 与 row_cache_u4

从汇编上看, `row_cache_u4` 的主要目标是进一步减少循环控制开销。在手工展开后, 单次循环体处理的元素数更多, 理论上可以减少索引更新和边界判断的比例。在高优化选项下, 普通的 `row_cache` 已经被编译器自动做了较强优化, 因此两者生成的汇编差距不会像 `-O0` 时那样明显。这也与性能测试相吻合: `row_cache_u4` 大多数情况下比 `row_cache` 更快, 但提升幅度远小于“按列访问改成按行访问”带来的收益。

因此, 矩阵题的 Godbolt 结论可以概括为: 决定性优化首先来自访问顺序的改变, 循环展开属于次一级优化; 编译器在 `-O3` 及以上会放大这种优势, 但无法从根本上修复不良的访存模式。

6.3 求和题汇编分析

6.3.1 sum_naive 与 sum_dual / sum_unroll14

求和题中最关键的现象是“累加依赖链”的变化。在 `sum_naive` 的 `-O3` 汇编中, 虽然编译器已经把每次迭代处理的数据量增加到多个元素, 但核心形式仍然是围绕单个标量累加寄存器反复执行 `addsd` 或 `vaddsd`, 即多个输入都串联到同一条累加链上。这样虽然减少了部分循环开销, 但依赖关系仍然较强。

`sum_dual` 和 `sum_unroll14` 的不同之处在于, 它们在源代码层面就构造了多条独立累加路径。尤其是 `sum_unroll14`, 在 `-O3` 下已经出现 `addpd` 对两路数据并行累加, 并在最后通过若干水平归约指令把多个局部和合并为最终结果; 而在 `-O3 -march=native` 下, 更进一步出现了 `vaddpd` 和 `ymm` 寄存器, 说明编译器利用 AVX 向量指令把原本四路展开的逻辑映射成了更宽的 SIMD 累加。

这说明 `sum_unroll14` 的优势并不仅仅是“少写了几次循环判断”, 更本质的原因是它让编译器看到多条彼此独立的累加链, 从而更容易同时利用超标量调度和向量化能力。这与表5中 `sum_dual`、`sum_unroll14` 在 `-O2/-O3` 下显著优于 `sum_naive` 的结果完全一致。

6.3.2 sum_unroll14 与 sum_pairwise

`sum_pairwise` 的汇编则体现出另一类优化思路: 它不是围绕单次扫描构建多累加器, 而是反复在数组上做“两两规约并写回”。从 Godbolt 汇编可以看到, 该函数包含多层循环结构, 并频繁执行从 `g_work` 读出两个元素、相加后再写回 `g_work` 的操作。即使在 `-O3 -march=native` 下, 这种“读-加-写回-再迭代”的结构仍然明显依赖内存访问。

因此, `sum_pairwise` 虽然在算法结构上有规约特征, 但当前实现方式需要大量中间结果写回, 难以像 `sum_unroll14` 那样把主要工作保留在寄存器中完成。这也解释了为什么它在实际测试中, 尤其是大规模下远慢于 `sum_dual` 和 `sum_unroll14`。

6.4 Godbolt 分析结论

结合以上汇编观察, 可以得到如下结论:

1. 对矩阵题而言, `row_cache` 的关键优势首先来自顺序访存, 而不是单纯依赖编译器自动优化; `-O3` 和 `-march=native` 只是把这种算法层面的优势进一步放大。
2. 对求和题而言, `sum_dual` 和 `sum_unroll14` 的核心收益来自减少单一累加器上的串行依赖, 使编译器更容易生成多路并行累加和 SIMD 指令。
3. `sum_pairwise` 说明“算法形式不同”并不必然意味着更快; 如果实现中引入了大量中间结果写回, 访存开销会抵消其结构优势。
4. Godbolt 所展示的汇编现象与前文性能表中的规律基本一致, 因此可以作为解释实验结果的底层证据之一。